Smart Traffic Management Data Pipeline and Analytics System

NAME : PARAMBRATA ACHARJEE
ROLL NO.:23051926
Data Engineering(IS30058)

Problem Statement

Modern traffic systems generate continuous streams of data from multiple sources such as sensors, vehicles, and traffic signals. However, this data is often unstructured and lacks proper processing pipelines, making it difficult to extract meaningful insights.

Key challenges include:

- Absence of real-time data processing
- Lack of structured storage systems
- Difficulty handling large datasets
- No centralized analytics platform
- Limited visualization capabilities

Proposed Solution

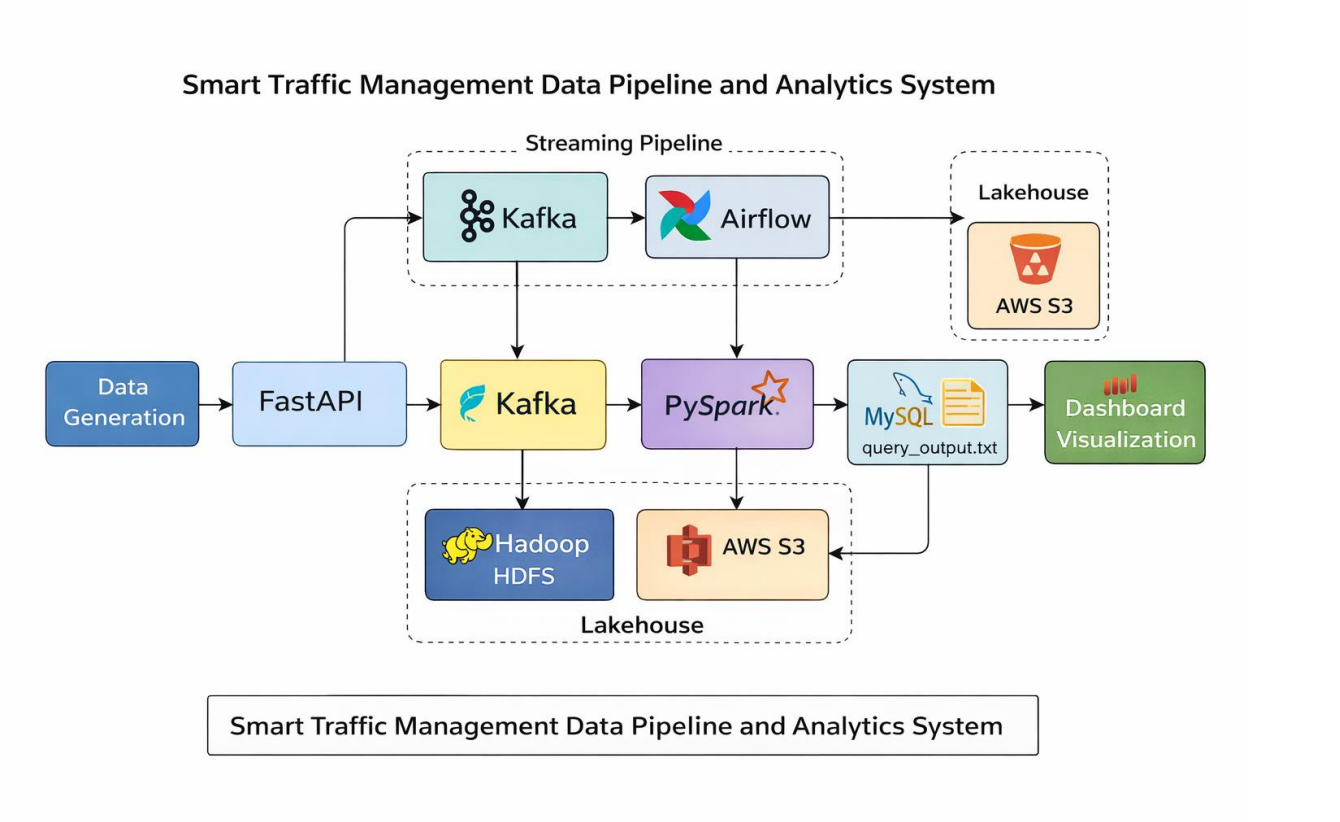
To address these challenges, a scalable end-to-end data engineering pipeline was developed.

The system performs:

- Synthetic traffic data generation
- Real-time streaming using Kafka
- Batch processing using Spark
- Workflow orchestration using Airflow
- Storage using Hadoop (HDFS) and AWS S3
- Querying using MySQL
- Schema modeling using Star Schema and Lakehouse architecture
- Visualization using Streamlit dashboard

- API integration using FastAPI
- ML model for predictive analysis

System Architecture Overview



Implementation Details

1. Data Generation

- Synthetic traffic dataset generated using Python
- Includes attributes like location, timestamp, speed, and vehicle count

2. Data Ingestion & API Layer (FastAPI)

- FastAPI used to expose endpoints
- Enables structured ingestion of traffic data

3. Streaming Pipeline (Kafka)

- Kafka producer simulates real-time traffic data

```
🚀 Producer started...
Sent: {'location': 'Junction_B', 'vehicle_count': 24, 'avg_speed':
 51.63, 'weather': 1, 'hour': 22}
Sent: {'location': 'Junction_B', 'vehicle_count': 71, 'avg_speed':
 35.64, 'weather': 0, 'hour': 6}
Sent: {'location': 'Junction_A', 'vehicle_count': 67, 'avg_speed':
 56.39, 'weather': 0, 'hour': 17}
Sent: {'location': 'Junction_B', 'vehicle_count': 13, 'avg_speed':
 15.32, 'weather': 0, 'hour': 5}
Sent: {'location': 'Junction_D', 'vehicle_count': 61, 'avg_speed':
 68.7, 'weather': 2, 'hour': 20}
Sent: {'location': 'Junction_B', 'vehicle_count': 25, 'avg_speed':
 40.45, 'weather': 0, 'hour': 14}
Sent: {'location': 'Junction_A', 'vehicle_count': 33, 'avg_speed':
 79.76, 'weather': 1, 'hour': 17}
Sent: {'location': 'Junction_B', 'vehicle_count': 44, 'avg_speed':
 15.28, 'weather': 2, 'hour': 7}
Sent: {'location': 'Junction_A', 'vehicle_count': 92, 'avg_speed':
 15.99, 'weather': 1, 'hour': 8}
```

- Kafka consumer processes streaming data

```
📥 Consumer started...
Inserted: {'location': 'Junction_D', 'vehicle_count': 94, 'avg_spe
ed': 52.9, 'weather': 0, 'hour': 9}
Inserted: {'location': 'Junction_D', 'vehicle_count': 18, 'avg_spe
ed': 22.33, 'weather': 1, 'hour': 13}
Inserted: {'location': 'Junction_C', 'vehicle_count': 64, 'avg_spe
ed': 71.06, 'weather': 0, 'hour': 14}
Inserted: {'location': 'Junction_C', 'vehicle_count': 27, 'avg_spe
ed': 13.28, 'weather': 2, 'hour': 8}
Inserted: {'location': 'Junction_C', 'vehicle_count': 13, 'avg_spe
ed': 70.72, 'weather': 1, 'hour': 6}
Inserted: {'location': 'Junction_C', 'vehicle_count': 85, 'avg_spe
ed': 51.53, 'weather': 0, 'hour': 23}
```

4. Data Processing (PySpark)

- Spark DataFrames used for large-scale processing
- Transformations, filtering, and aggregations applied

- Spark SQL used for querying

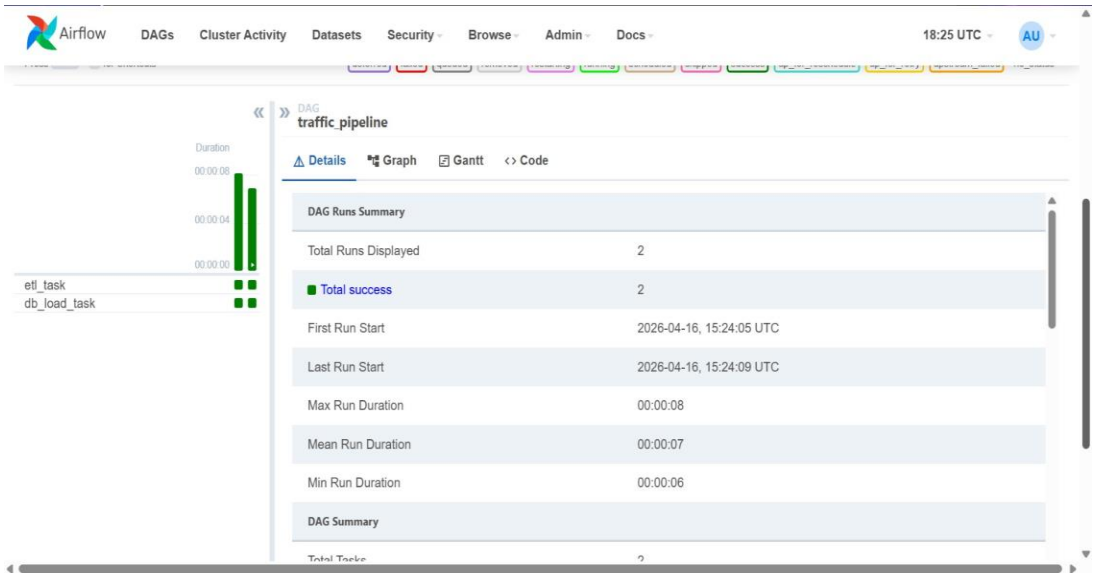
```
✅ Spark Session Created
+-------------------+----------+------------+---------+-------+--------+----------+----+---------+-------
------+
|          timestamp|  location|vehicle_count|avg_speed|weather|accident|congestion|hour|      day|traffic
_level|
+-------------------+----------+------------+---------+-------+--------+----------+----+---------+-------
------+
|2026-04-14 23:53:...|Junction_B|          18|    48.53|  Clear|       0|         0|  23|  Tuesday|
   Low|
|2026-04-15 02:25:...|Junction_A|          32|    50.22|  Clear|       0|         0|   2|Wednesday|
   Low|
|2026-04-15 06:30:...|Junction_B|          22|    47.81|    Fog|       1|         1|   6|Wednesday|
   Low|
|2026-04-15 02:36:...|Junction_D|          33|     50.7|  Clear|       0|         0|   2|Wednesday|
   Low|
|2026-04-15 00:15:...|Junction_A|          49|    37.07|    Fog|       0|         0|   0|Wednesday|
Medium|
+-------------------+----------+------------+---------+-------+--------+----------+----+---------+-------
------+
only showing top 5 rows
```

```
📊 Traffic by location:
+----------+------------------+--------+
|  location|       avg_vehicles|records|
+----------+------------------+--------+
|Junction_A| 52.79936305732484|    1256|
|Junction_D|  52.7341873498799|    1249|
|Junction_C|53.534995977473855|    1243|
|Junction_B| 52.17571884984026|    1252|
+----------+------------------+--------+

📈 Spark SQL Result:
+----------+------------------+
|  location|        avg_traffic|
+----------+------------------+
|Junction_C|53.534995977473855|
|Junction_A| 52.79936305732484|
|Junction_D|  52.7341873498799|
|Junction_B| 52.17571884984026|
+----------+------------------+
```

5. Workflow Orchestration (Airflow)

- DAG automates:
  - Data ingestion
  - Processing
  - Storage



6. Hadoop (HDFS)

- Distributed storage of processed data
- Enables scalable data handling

```
(venv) kiit@BT01770:~/smart-traffic-system/hadoop/hadoop-3.3.6$ jps
192497 NameNode
192887 SecondaryNameNode
192628 DataNode
193103 Jps
(venv) kiit@BT01770:~/smart-traffic-system/hadoop/hadoop-3.3.6$ hdfs dfs -mkdir /traffic

hdfs dfs -put ~/smart-traffic-system/data/processed/traffic_data_cleaned.csv /traffic

hdfs dfs -ls /traffic
Found 1 items
-rw-r--r--   1 kiit supergroup     367915 2026-04-17 02:46 /traffic/traffic_data_cleaned.csv
```

7. Lakehouse Architecture

- Data stored in Parquet format
- Combines data lake flexibility with warehouse structure

```
📥 Raw Data
+--------------------+----------+-------------+---------+-------+--------+----------+----+---------+-------------+
|           timestamp|  location|vehicle_count|avg_speed|weather|accident|congestion|hour|      day|traffic_level|
+--------------------+----------+-------------+---------+-------+--------+----------+----+---------+-------------+
|2026-04-14 23:53:...|Junction_B|           18|    48.53|  Clear|       0|         0|  23|  Tuesday|          Low|
|2026-04-15 02:25:...|Junction_A|           32|    50.22|  Clear|       0|         0|   2|Wednesday|          Low|
|2026-04-15 06:30:...|Junction_B|           22|    47.81|    Fog|       1|         1|   6|Wednesday|          Low|
|2026-04-15 02:36:...|Junction_D|           33|     50.7|  Clear|       0|         0|   2|Wednesday|          Low|
|2026-04-15 00:15:...|Junction_A|           49|    37.07|    Fog|       0|         0|   0|Wednesday|       Medium|
+--------------------+----------+-------------+---------+-------+--------+----------+----+---------+-------------+
only showing top 5 rows
✅ Data stored in lakehouse format (Parquet)
📊 Querying Lakehouse Data
+----------+------------------+
|  location|avg(vehicle_count)|
+----------+------------------+
|Junction_A| 52.79936305732484|
|Junction_D|  52.7341873498799|
|Junction_C|53.534995977473855|
|Junction_B| 52.17571884984026|
+----------+------------------+
```

8. Star Schema Design

- Fact Table: traffic metrics
- Dimension Tables: time, location
- Optimized for analytical queries
- Output stored in: ->data/star

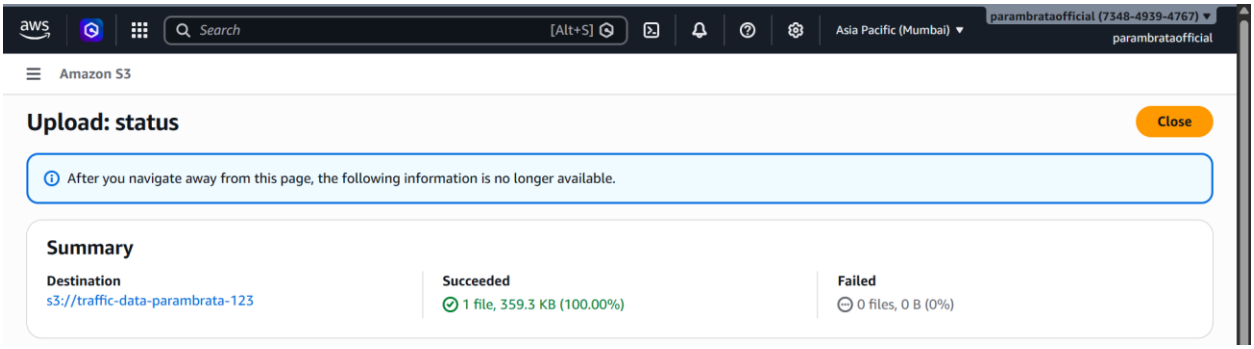
9. Database Integration (MySQL)

- Data loaded into MySQL
- Aggregation Queries (AVG, SUM, COUNT)
- Time-based Analysis using HOUR()
- Multi-table Joins (Star Schema)
- Grouped Analytical Queries
- Data Filtering and Sorting
- Derived Metrics Computation
- Output stored in: ->db/query_output.txt

10. Cloud Integration

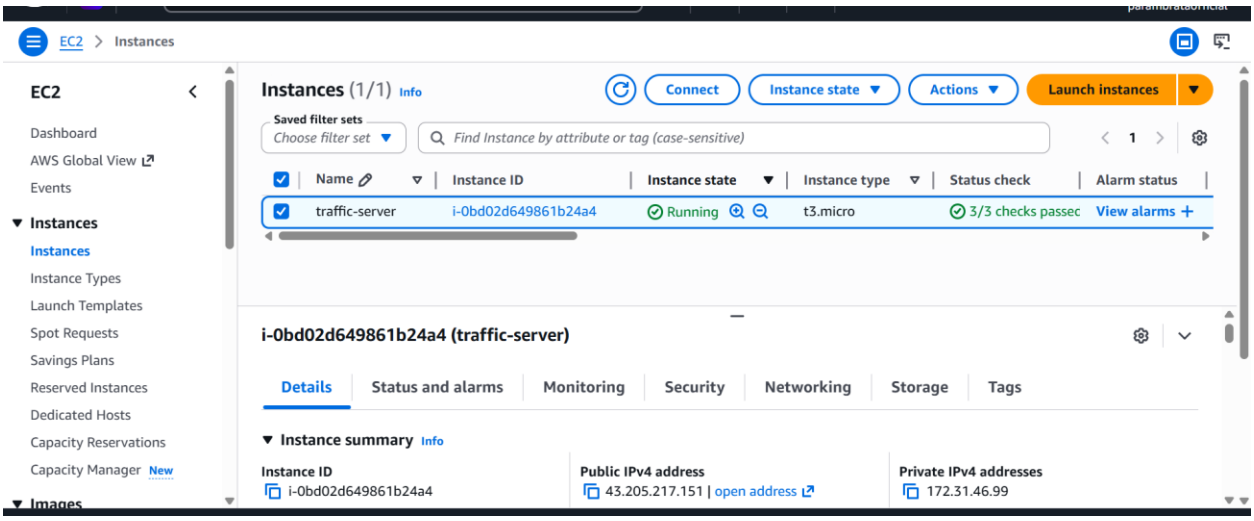
AWS S3

- Dataset uploaded for storage



AWS EC2

- Pipeline deployed and executed



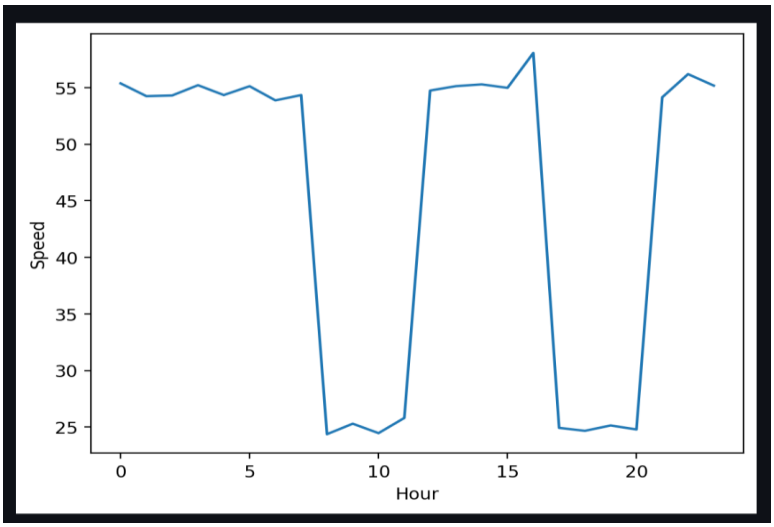
11. Machine Learning Integration

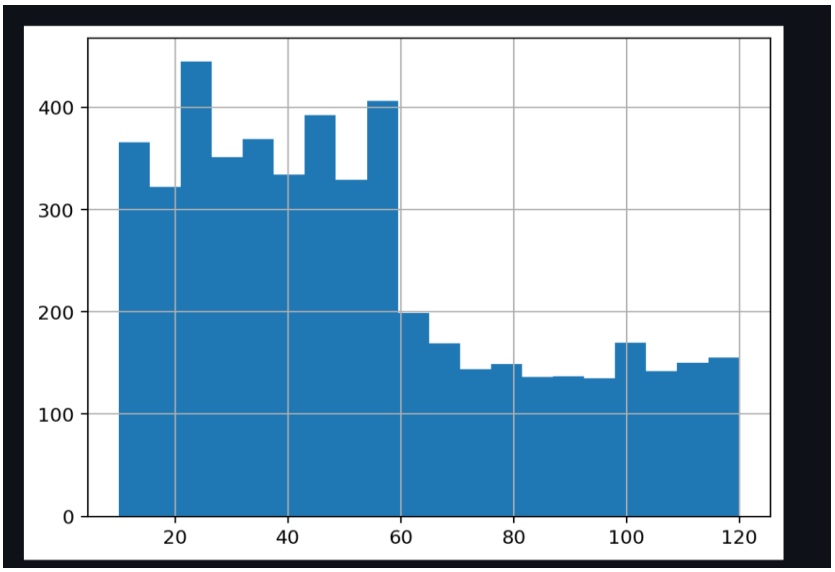
- Model trained on traffic data
- Used for predictive insights

12. Dashboard Visualization (Streamlit)

Dashboard features:

- Traffic by location
- Speed trends
- Vehicle distribution





Tech Stack

- Python
- Apache Kafka
- Apache Spark
- Apache Airflow
- Hadoop (HDFS)
- MySQL
- AWS (S3, EC2)
- Streamlit
- FastAPI

Unique Contributions

- End-to-end pipeline implementation
- Integration of batch + streaming systems
- Star schema + Lakehouse architecture
- Cloud deployment
- API + ML integration

Conclusion

This project demonstrates a complete and scalable data engineering solution capable of handling traffic data, ensuring data quality, and generating actionable insights using modern tools and cloud infrastructure.

Future Improvements

- Implement real-time processing using **Spark Streaming with Kafka**
- Deploy the Streamlit dashboard on cloud (AWS/Render) for public access
- Integrate with data warehouses like **Amazon Redshift or Google BigQuery**
- Enhance the lakehouse using **Delta Lake or Apache Iceberg**
- Add **machine learning models** for traffic prediction and congestion detection
- Implement **alert systems** for real-time traffic anomalies
- Optimize performance using **partitioning and indexing techniques**
- Enable **real-time API-based data ingestion** from live traffic sources
- Improve data visualization with advanced dashboards and KPIs
- Automate the pipeline using CI/CD for continuous deployment